

Quantum Operator Spreading and Krylov Complexity

UA

June 26, 2025

Agenda

- Module overview and motivation
- Pauli basis decomposition
- Symmetry constraints and basis reduction
- Operator evolution and spreading measures
- Krylov subspace construction
- Gram-Schmidt orthogonalization
- Krylov complexity calculation
- Implementation and visualization
- Conclusions

Module Overview

Goal: Study operator spreading in quantum circuits using Krylov complexity

Key Features:

- Pauli basis decomposition with symmetry reduction
- Systematic Krylov subspace construction
- Gram-Schmidt orthogonalization procedure
- Multiple spreading diagnostics and visualization tools

Applications: Quantum chaos, thermalization, circuit complexity

Pauli Basis Decomposition

Any operator on n qubits can be written as:

$$O = \sum_{P \in \mathcal{P}_n} c_P P \quad (1)$$

where $\mathcal{P}_n = \{I, X, Y, Z\}^{\otimes n}$ is the full Pauli basis

The coefficients are given by:

$$c_P = \frac{1}{2^n} \text{Tr}[OP] \quad (2)$$

Symmetry Constraints: $\mathbb{Z}_2$ Symmetry

Parity symmetry: Circuits invariant under $\prod_i \sigma_i^z$

Constraint: Only Pauli strings with even number of X and Y

$$\mathcal{P}_n^{\mathbb{Z}_2} = \{P \in \mathcal{P}_n : n_X(P) + n_Y(P) \equiv 0 \pmod{2}\} \quad (3)$$

Basis size reduction:

$$|\mathcal{P}_n^{\mathbb{Z}_2}| = 2^{2n-1} = \frac{4^n}{2} \quad (4)$$

Factor of 2 reduction in computational cost!

Symmetry Constraints: $U(1)$ Symmetry

Charge conservation: Circuits invariant under $\exp(i\phi \sum_j \sigma_j^z)$

Constraint: Equal numbers of X and Y operators

$$\mathcal{P}_n^{U(1)} = \{P \in \mathcal{P}_n : n_X(P) = n_Y(P)\} \quad (5)$$

Basis size:

$$|\mathcal{P}_n^{U(1)}| = \sum_{k=0}^{\lfloor n/2 \rfloor} \binom{n}{2k} \binom{2k}{k} 2^{n-2k} \approx \frac{4^n}{\sqrt{\pi n}} \quad (6)$$

Factor of $\sqrt{\pi n}$ reduction!

Basis Size Comparison

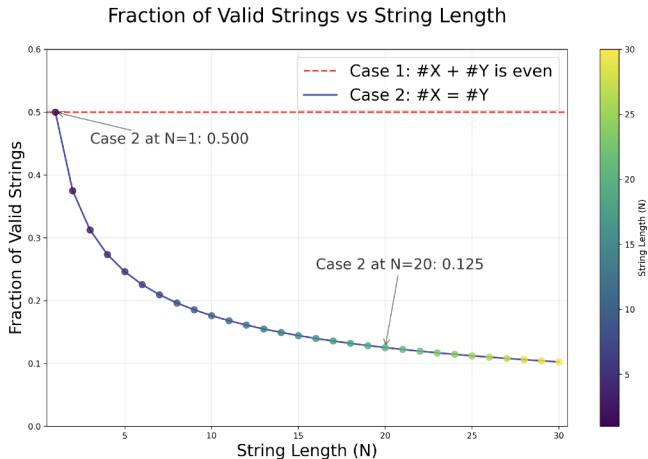


FIG. 1: Comparison of the string length reduction under the symmetries considered

Operator Evolution

Unitary dynamics: Operators evolve under superoperator

$$O(t) = \mathcal{U}_t[O_0] \quad \text{where} \quad \mathcal{U}[\cdot] = U^\dagger(\cdot)U \quad (7)$$

Weight coefficients: For $O(t) = \sum_i w_i(t)P_i$

$$w_i(t) = \frac{1}{2^n} \text{Tr}[O(t)P_i] \quad (8)$$

Evolution spreads operator support across many Pauli strings

Spreading Measures

Participation Ratio:

$$\text{PR}(t) = \frac{1}{\sum_P |c_P(t)|^4} \quad (9)$$

Average Weight:

$$\langle w \rangle(t) = \sum_{w=0}^n w \cdot \rho_w(t) \quad (10)$$

where $\rho_w(t) = \sum_{P: \text{wt}(P)=w} |c_P(t)|^2$ and
 $\text{wt}(P) = n_X(P) + n_Y(P) + n_Z(P)$

Krylov Subspace: Motivation

Problem: How to quantify operator complexity beyond simple spreading?

Solution: Construct orthogonal basis from operator evolution sequence

Operator sequence:

$$|O_t\rangle = \mathcal{U}^t |O_0\rangle, \quad t = 0, 1, 2, \dots \quad (11)$$

Inner product:

$$\langle A|B\rangle = \frac{1}{2^n} \text{Tr}[A^\dagger B] \quad (12)$$

Gram-Schmidt Orthogonalization: Setup

Goal: Find orthonormal basis $\{|K_0\rangle, |K_1\rangle, |K_2\rangle, \dots\}$

Step 0: Normalize initial operator

$$|K_0\rangle = \frac{|O_0\rangle}{\sqrt{\langle O_0|O_0\rangle}} \quad (13)$$

General procedure: Remove projections onto previous basis vectors

$$|A_n\rangle = |O_n\rangle - \sum_{j=0}^{n-1} \langle K_j|O_n\rangle |K_j\rangle \quad (14)$$

$$|K_n\rangle = \frac{|A_n\rangle}{\sqrt{\langle A_n|A_n\rangle}} \quad (15)$$

Gram-Schmidt: First Step Example

Initial operator: $|O_0\rangle = \sum_i w_i(0)P_i$

Normalized:

$$|K_0\rangle = \frac{\sum_i w_i(0)P_i}{\sqrt{\sum_i |w_i(0)|^2}} \quad (16)$$

After one evolution: $|O_1\rangle = \sum_i w_i(1)P_i$

Orthogonalize:

$$|A_1\rangle = |O_1\rangle - \langle K_0|O_1\rangle|K_0\rangle \quad (17)$$

Gram-Schmidt: Second Step

Overlap calculation:

$$\langle K_0 | O_1 \rangle = \frac{\sum_j w_j(1) w_j^*(0)}{\sqrt{\sum_k |w_k(0)|^2}} \quad (18)$$

Orthogonal component:

$$|A_1\rangle = \sum_i w_i(1) P_i - \frac{\sum_j w_j(1) w_j^*(0)}{\sqrt{\sum_k |w_k(0)|^2}} |K_0\rangle \quad (19)$$

Normalize:

$$|K_1\rangle = \frac{|A_1\rangle}{||A_1||} \quad \text{if } ||A_1|| > \epsilon \quad (20)$$

Krylov Complexity Definition

Expansion in Krylov basis:

$$|O_t\rangle = \sum_{n=0}^{D_K-1} \beta_{t,n} |K_n\rangle \quad (21)$$

Krylov complexity:

$$K(t) = \sum_{n=0}^{D_K-1} n |\beta_{t,n}|^2 \quad (22)$$

Measures how much the operator has "spread" in the Krylov subspace

Maximally Ergodic Regime

Superoperator matrix elements:

$$a_n = \langle K_n | \mathcal{U} | K_n \rangle \quad (\text{diagonal}) \quad (23)$$

$$b_n = \langle K_n | \mathcal{U} | K_{n-1} \rangle \quad (\text{subdiagonal}) \quad (24)$$

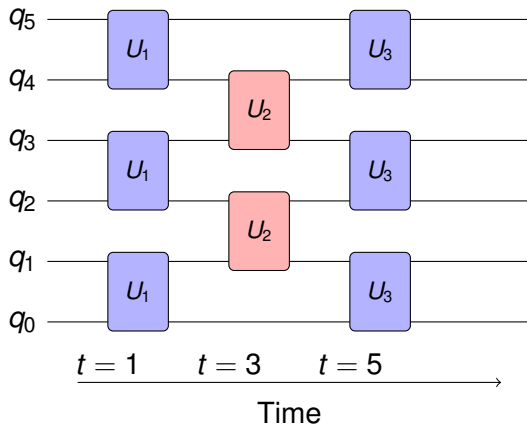
Ergodic conditions:

$$a_n \rightarrow 0 \quad (\text{diagonal elements vanish}) \quad (25)$$

$$b_n \rightarrow 1 \quad (\text{subdiagonal elements saturate}) \quad (26)$$

$$K(t) \rightarrow t \quad (\text{linear complexity growth}) \quad (27)$$

Brickwork Circuit Implementation



Alternating even/odd two-qubit gates: $\mathcal{U}_t[\cdot] = U_t^\dagger(\cdot)U_t$

Key Implementation Functions

Pauli string to matrix:

```
def pauli_string_to_matrix(self, pauli_string):  
    result = self.pauli_matrices[pauli_string[0]]  
    for char in pauli_string[1:]:  
        result = np.kron(result,  
self.pauli_matrices[char])
```

Overlap computation:

```
def compute_overlap(self, operator):  
    overlaps = []  
    for pauli_string in self.pauli_basis:  
        pauli_matrix =  
self.pauli_string_to_matrix(pauli_string)  
        overlap = (1/2**n) * np.trace(operator @  
pauli_matrix)  
        overlaps.append(overlap)
```

Visualization Tools

- **Pauli coefficients heatmap:** $|c_P(t)|$ evolution
- **Weight distribution:** $\rho_w(t)$ vs time
- **Krylov complexity:** $K(t)$ growth
- **Participation ratio:** Spreading measure
- **Krylov basis coefficients:** $|\beta_{t,n}|^2$
- **Basis size comparison:** Symmetry scaling

All functions designed for easy interpretation of complex quantum dynamics

Work in Progress: Superoperator Analysis

Hessenberg matrix structure: Superoperator in Krylov basis

$$\mathcal{U}_{mn} = \begin{cases} 0 & \text{if } m > n + 1 \\ b_m & \text{if } m = n + 1 \\ a_m \frac{c_m}{c_n} & \text{if } m \leq n \end{cases} \quad (28)$$

Future directions:

- Connection to quantum chaos indicators
- Thermalization diagnostics

What's next for computation?

Unitary superoperator (order of 1-2 days)

Non-markovian circuits

- Code format takes list of unitaries and applies it cyclically if $\text{len}(\text{list}) \leq \text{len}(\text{circuit time steps})$
- Provide NM circuit $\rightarrow$ look at operator spreading
- Two qubit $U(1)$ and $Z_2 \implies$ we can still extract map parameters $\implies$ analyze map parameter
- @Pei-jun wrote a module on visualizing area vs volume law transition.
- Code has ability to save upto 8Q DM. Area vs Vol law analysis

What's next in theory

NM circuits and revivals in KC

- NM measure and how to apply to circuit structure
- How can we engineer constrained dynamics at the level of KC
- What does constrained dynamics in KC look like for map parameters
- How can circuit structure constraints generate non-uniform Pauli operator weight distribution
- For area law, need situation where nearest neighbour correlation weight is high compared to long distance correlation weight
- Can we have cycles in KC?

What we have seen

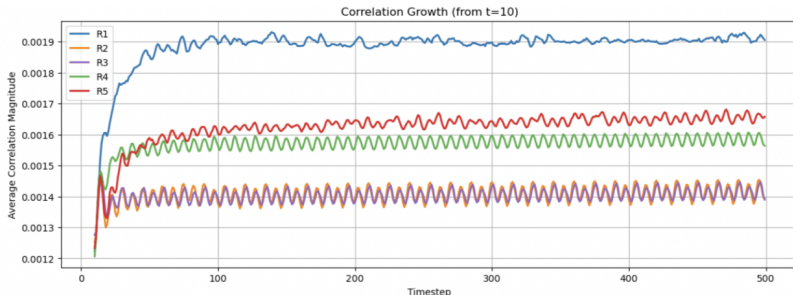


Figure: Depending on update rule ie type of circuit constraint, the correlation growth saturates at different value.

Similar plot for $K(t)$ would potentially allow distinguishing between regimes of strong scrambling (high saturation) and weak scrambling or localization (low saturation). Here, the specific value reflects how much of the basis is explored by the evolving operator

Conclusions

what's done:

- Efficient operator spreading simulation with symmetry reduction
- Systematic Krylov complexity calculation
- Comprehensive spreading diagnostics
- Robust computational framework

what can we study:

- Quantum thermalization studies
- Circuit complexity optimization
- Quantum chaos diagnostics